

# ATIMS Pipeline - Code & Implementation Architecture

## 1. Purpose

This is the document for the AP taxability classification model and the pipeline built around it.

The model's purpose is to classify AP invoice line items for taxability. For each line it determines the category, looks up whether that category is taxable in its state, flags any required use-tax reversal, and produces an audit-ready justification - escalating uncertain cases for human review. The existing trained model is preserved unchanged (TF-IDF features + XGBoost/LightGBM ensemble + DBFS artifacts), with the LLM steps added through the Databricks Foundation Model API functions `ai_classify()` and `ai_query()`.

The value it delivers: most lines are classified, taxed, justified, and quality-checked automatically; only genuinely uncertain lines reach a human, and every line carries a written justification and a full trace of how it was decided.

## 2. How it works (overview)

The pipeline runs monthly as a **9-task Databricks Workflow**. The tasks form a straight, linear chain. They do **not** share in-memory DataFrames - each task reads a Delta staging table and writes the next, so the order is fixed, each stage is independently re-runnable, and every intermediate result is inspectable.

```
t30 ingest/prep -> t31 rules expert -> t32 classifier -> t33 tax lookup ->
t33b use-tax -> t34 tax writer -> t35 QA -> t36 reroute/finalize -> t37 write+notify
```

The logic flows in three movements:

- 1. **Classify** - rules first (cheap, deterministic), then an ML + LLM ensemble for whatever the rules don't cover (t31-t32).
- 2. **Determine tax** - taxability from a reference matrix, then use-tax reversal logic, then a written justification (t33-t34).
- 3. **Assure & persist** - quality checks, escalation of anything suspicious, and the final write plus notification (t35-t37).

## 3. Repository layout

| Path                   | Role                                                                                                                                                                                                |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| atims_common.py        | Shared module: configuration + the model layer (rules engine, TF-IDF feature builder, artifact loader, ensemble predictor) + Delta/LLM helpers. Imported by every task; <b>not</b> a workflow step. |
| t30_ingest_prep.py     | Read the fact table, filter to the run month, build <code>final_description</code> .                                                                                                                |
| t31_rules_expert.py    | Rules engine -> tag each line <code>rule</code> or <code>no_rule</code> .                                                                                                                           |
| t32_classifier.py      | Existing ensemble + <code>ai_classify</code> + <code>ai_query</code> resolution.                                                                                                                    |
| t33_tax_lookup.py      | Taxability matrix join.                                                                                                                                                                             |
| t33b_use_tax_lookup.py | <code>Reverse_UseTax</code> determination.                                                                                                                                                          |
| t34_tax_writer.py      | <code>ai_query</code> audit write-ups.                                                                                                                                                              |
| t35_qa.py              | Six quality checks + anomaly scoring.                                                                                                                                                               |
| t36_reroute.py         | Escalation / finalize.                                                                                                                                                                              |
| t37_write_output.py    | Final write + MLflow metrics + report + webhook.                                                                                                                                                    |
| atims_workflow.json    | Databricks Jobs spec wiring the 9 tasks with <code>depends_on</code> .                                                                                                                              |
| README.md              | Setup and wiring notes.                                                                                                                                                                             |
| docs/                  | This document (markdown + PDF).                                                                                                                                                                     |

## 4. Execution model & staging tables

Each task reads one Delta table and writes one Delta table, via the helpers `ac.read_stage()` / `ac.write_stage()`. The table names live in `atims_common.T`:

| Step | Reads                          | Writes                         | Carries forward (key additions)                                                                                                                                                              |
|------|--------------------------------|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| t30  | <code>VMAP_FACT_TABLE</code>   | <code>t30_prepped</code>       | <code>final_description</code>                                                                                                                                                               |
| t31  | <code>t30_prepped</code>       | <code>t31_rules_labeled</code> | <code>category</code> , <code>rule_matched</code> , <code>label_account_code</code> , <code>label_source</code>                                                                              |
| t32  | <code>t31_rules_labeled</code> | <code>t32_classified</code>    | <code>final_pred</code> , <code>final_conf</code> , <code>llm_pred</code> , <code>agreement</code> , <code>final_category</code> , <code>confidence_tier</code> , <code>decision_path</code> |
| t33  | <code>t32_classified</code>    | <code>t33_tax_lookup</code>    | <code>taxability</code> , <code>tax_source</code> , <code>review_priority_pre</code>                                                                                                         |
| t33b | <code>t33_tax_lookup</code>    | <code>t33b_use_tax</code>      | <code>use_tax_amount</code> , <code>Reverse_UseTax</code>                                                                                                                                    |
| t34  | <code>t33b_use_tax</code>      | <code>t34_tax_writer</code>    | <code>short_justification</code> , <code>full_writeup</code> , <code>audit_ready</code>                                                                                                      |
| t35  | <code>t34_tax_writer</code>    | <code>t35_qa</code>            | <code>anomalies</code> , <code>anomaly_score</code> , <code>review_priority</code> , <code>override_suggested</code> , <code>recommendation</code>                                           |
| t36  | <code>t35_qa</code>            | <code>t36_rerouted</code>      | <code>iteration</code> , <code>processing_path</code> , <code>escalated_label_source</code>                                                                                                  |
| t37  | <code>t36_rerouted</code>      | <code>PROD_TABLE</code>        | final selected schema                                                                                                                                                                        |

Because the hand-off is through tables, you can re-run from any point (e.g. re-run t34 onward) without redoing earlier stages, as long as the upstream staging tables still exist.

## 5. The shared module ( `atims_common.py` )

Everything common lives here so the task notebooks stay thin and the model code is provably unchanged.

**Configuration** - catalog/schema and staging-table names; the production output table; the taxability matrix and optional QA reference tables; the LLM endpoint; the `ai_classify` label set; the account->category map; the webhook; the model thresholds; and the DBFS paths to the source table and model artifacts.

**The model layer (ported verbatim from the prior production notebook):**

| Function                                                                                                                                                             | Role                                                                                              |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| <code>load_rules</code> , <code>_apply_keyword_rules</code> , <code>_apply_generic_rules</code> , <code>_apply_parker_volpe_rules</code> , <code>build_labels</code> | The rules engine: load the rule sheets and apply the three tiers.                                 |
| <code>build_final_description</code>                                                                                                                                 | Combine the description columns into clean text.                                                  |
| <code>build_feature_matrix</code> , <code>_fit_tfidf</code> , <code>_transform_tfidf</code> , <code>_hash_vendor</code>                                              | Feature engineering (TF-IDF + categoricals + vendor hash + amount).                               |
| <code>load_artifacts</code>                                                                                                                                          | Load the XGBoost, LightGBM, label encoder, categorical encoders, and TF-IDF vectorizer from DBFS. |
| <code>predict_no_rule</code>                                                                                                                                         | Run the ensemble on <code>no_rule</code> rows and return raw predictions.                         |
| <code>generate_report</code>                                                                                                                                         | Builds the need-to-review Excel report used by t37.                                               |

**Helpers** - `get_spark` / `get_dbutils` , `read_stage` / `write_stage` , and the `ai_classify` label-array / prompt-label builders.

The model code is untouched; only the post-prediction routing was moved out of `predict_no_rule` into t32 so the agreement logic lives with the classifier task.

## 6. Pipeline tasks in detail

### t30 - Ingest & prep

Reads `VMAP_FACT_TABLE` , filters to `ACCOUNTING_DATE LIKE '<month>%'` , and builds `final_description` by coalescing `LINE_DESCRIPTION` / `DESCRIPTION` / `DIST_ALL_DESCRIPTION` , stripping filler values (e.g. "SCM Purchase"). Priority is line+description, then line, then description, then distribution text. **Output:** `t30_prepped` .

### t31 - Rules expert

Applies the three rule tiers in order - **keyword, generic, Parker-Volpe** - each overwriting the previous (later match wins). Generic rules cover the deterministic EXTC/MIC/account/vendor/company patterns; Parker-Volpe covers vendor- and description-specific rules loaded from the rules workbook. Matched lines get a category, a `rule_matched` tag, and a `label_account_code` (mapped from that category, or the raw `ACCOUNT_CODE` ), and are tagged `label_source = 'rule'` . Everything else is `'no_rule'` . Deterministic rules only; whatever a rule does not match is handled downstream by the classifier. **Output:** `t31_rules_labeled` .

### t32 - Classifier

Operates on `no_rule` rows. Rule rows pass straight through with `final_category` set from the rule, `final_conf = 1.0` , `confidence_tier = RULE` , `decision_path = rule` . The full decision logic is in Section 8. **Output:** `t32_classified` .

### t33 - Tax lookup

Resolves `final_category` (falling back to the ensemble prediction when needed) and LEFT-joins the taxability matrix on `final_category + state` , yielding `taxability` (Taxable | Exempt | null) and `tax_source` ( `matrix` | `no_match` ). A null taxability sets `review_priority_pre = HIGH` (unmatched goes straight to human review - there is no model guessing taxability). **Output:** `t33_tax_lookup` .

### t33b - Use-tax lookup

Pure conditional logic producing `Reverse_UseTax` (see Section 9). **Output:** `t33b_use_tax` .

### t34 - Tax writer

Calls `ai_query()` once per row to generate `full_writeup` , an audit-ready justification assembled from the structured facts (category, state, taxability, amounts, basis). When `Reverse_UseTax = 'Yes'` , the prompt forces the write-up to state the reversal requirement and reference the amount. `short_justification` is a lightweight template (no extra LLM call); `audit_ready = (taxability is not null)` . **Output:** `t34_tax_writer` .

### t35 - QA

Runs six Spark checks and rolls them into an anomaly score and review priority (see Section 10). **Output:** `t35_qa` .

### t36 - Re-route / finalize

Sets `iteration = 0` , escalates `override_suggested` rows to `label_source = 'need_to_review'` , and labels `processing_path` (FAST / STANDARD / COMPLEX / ESCALATION). Re-running the deterministic ensemble would produce the same answer, so this task finalizes and routes exceptions to humans rather than looping. **Output:** `t36_rerouted` .

### t37 - Write output + notify

Selects the final schema, appends to `PROD_TABLE` (partitioned by `ACCOUNTING_DATE` ), logs batch metrics to the existing MLflow experiment, generates the need-to-review Excel report, and POSTs a completion webhook to the FastAPI backend. **Output:** `PROD_TABLE` .

## 7. Feature engineering & the model (unchanged)

Each line is turned into a single feature vector by `build_feature_matrix`:

| Block        | Source                                                                                                                   | Dimensions   | Notes                                                             |
|--------------|--------------------------------------------------------------------------------------------------------------------------|--------------|-------------------------------------------------------------------|
| Text         | TF-IDF of <code>final_description</code>                                                                                 | up to 10,000 | word uni+bigrams, sublinear TF, <code>min_df=2</code>             |
| Categoricals | <code>EXTC_CODE</code> , <code>MIC</code> , <code>COMPANY_CODE</code> , <code>STATE</code> , <code>INVOICE_SOURCE</code> | 5            | <code>LabelEncoder</code> ; unseen values -> <code>UNKNOWN</code> |
| Vendor       | <code>VENDOR_NAME</code>                                                                                                 | 1            | MD5 hash into 5,000 bins                                          |
| Amount       | <code>TRAN_AMOUNT</code>                                                                                                 | 1            | <code>log1p(abs(amount))</code>                                   |

These are concatenated into roughly a 10,007-dimension vector. The classifier is an ensemble:

- **XGBoost** (`hist`) and **LightGBM** each predict class probabilities.
- Probabilities are averaged **50/50**: `ens = 0.5*xgb + 0.5*lgb`.
- The predicted class is `argmax(ens)`, mapped back to an **account code** via the label encoder; `final_conf` is that class's averaged probability.
- `models_agree` records whether XGBoost and LightGBM independently picked the same class.

Artifacts loaded from DBFS: `xgb_model.ubj`, `lgb_model.txt`, `label_encoder.pkl`, `cat_encoders.pkl`, `tfidf.pkl`. The pipeline **loads** these; it never retrains. MLflow experiment: `ap_tax_classification`.

## 8. Classifier decision logic (t32)

For `no_rule` rows the task runs two classifiers and an LLM tiebreaker:

1. **Sub-model A** - the existing ensemble (Section 7) -> `final_pred` (account code), `final_conf`.
2. **Map to category** - `model_category = ACCOUNT_TO_CATEGORY[final_pred]` if the map is configured, else `final_pred`.
3. **Sub-model B** - `ai_classify(final_description, array(<CATEGORY_LABELS>))` -> `llm_pred` (a category name).
4. **Compare** - `agreement = (model_category == llm_pred)`.

| Condition                                 | <code>final_category</code>           | <code>confidence_tier</code> | <code>decision_path</code> |
|-------------------------------------------|---------------------------------------|------------------------------|----------------------------|
| agree, <code>final_conf &gt; 0.80</code>  | <code>model_category</code>           | HIGH                         | STANDARD                   |
| agree, <code>final_conf &lt;= 0.80</code> | <code>model_category</code>           | MEDIUM                       | STANDARD                   |
| disagree                                  | <code>ai_query(...)</code> resolution | RESOLVED                     | COMPLEX                    |

The `ai_query` resolver receives both predictions plus the description and must return the single best label from the category list. `ensemble_confidence` is set to `final_conf`, and all `no_rule` rows are tagged `label_source = 'ml'`.

The ML predictions are joined back to the line records on `invoice_id + inv_line_number` (line grain), which prevents multi-line invoices from fanning out.

## 9. Taxability & use-tax determination

**Taxability (t33)** is a lookup, not a model output: `final_category + state` against the reference matrix. No match -> `taxability = null` -> the line is escalated (HIGH) for a human; the pipeline never infers taxability.

**Use-tax reversal (t33b)** is deterministic logic on the AP-system `use_tax_amount` and the resolved `taxability`:

| <code>use_tax_amount</code> | <code>taxability</code> | <code>Reverse_UseTax</code> | <code>Meaning</code>                                |
|-----------------------------|-------------------------|-----------------------------|-----------------------------------------------------|
| > 0                         | Exempt                  | <b>Yes</b>                  | use tax paid on an exempt line -> reversal required |
| > 0                         | Taxable                 | No                          | use tax correctly paid                              |
| = 0                         | any                     | No                          | no use tax present                                  |
| NULL                        | any                     | NULL                        | deferred - pending NULL-vs-data-gap confirmation    |
| any                         | NULL                    | NULL                        | taxability unresolved -> cannot determine           |

When `Reverse_UseTax = 'Yes'`, the Tax Writer (t34) is required to state the reversal and cite the amount in the justification.

## 10. Quality assurance: checks & scoring (t35)

Six checks run as Spark window / statistical operations:

| # | Check                       | Fires when                                                                                                   |
|---|-----------------------------|--------------------------------------------------------------------------------------------------------------|
| 1 | Vendor consistency          | vendor has >=5 lines this month and this category is the modal one for <50% of them                          |
| 2 | Amount outlier              | absolute z-score > 3 within the category (skipped if no variance)                                            |
| 3 | New state/category          | combination not seen historically (only if <code>HISTORICAL_STATS</code> is wired)                           |
| 4 | Rule vs classifier conflict | dormant - the classifier doesn't run on rule rows, so there's nothing to compare                             |
| 5 | Taxability sanity           | a service-type category marked Taxable in a no-service-tax state (only if <code>STATE_RULES</code> is wired) |
| 6 | Batch outlier               | vendor has >=10 lines and this category appears in <=5% of them                                              |

**Scoring.** Each fired check contributes equally: `anomaly_score = min( (#flags / 6) x 2, 1.0 )` - so 1 flag ~ 0.33, 2 ~ 0.67, 3+ = 1.0.

**Priority.** `review_priority` is, in order: **HIGH** if taxability was unmatched (set in t33, never downgraded); else **CRITICAL** >= 0.66; **HIGH** >= 0.33; **MED** > 0; otherwise **LOW**.

**Escalation.** `override_suggested = true` when `anomaly_score >= 0.33` or priority is HIGH/CRITICAL; this is what t36 escalates to the review queue.

## 11. Output data dictionary ( `PROD_TABLE` )

Written append-mode, partitioned by `accounting_date`.

| Column                                                                                                                    | Meaning                                                                             |
|---------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <code>invoice_id</code> , <code>inv_line_number</code>                                                                    | Line key (grain of the table)                                                       |
| <code>final_description</code>                                                                                            | Cleaned line text used for classification                                           |
| <code>vendor_name</code> , <code>state</code> , <code>tran_amount</code>                                                  | Carried-through line attributes                                                     |
| <code>final_category</code>                                                                                               | Resolved category (from a rule, agreement, or LLM resolution)                       |
| <code>taxability</code>                                                                                                   | <code>Taxable</code> / <code>Exempt</code> / null                                   |
| <code>tax_source</code>                                                                                                   | <code>matrix</code> / <code>no_match</code>                                         |
| <code>use_tax_amount</code> , <code>Reverse_UseTax</code>                                                                 | Use-tax inputs and the reversal decision                                            |
| <code>final_pred</code> , <code>final_conf</code>                                                                         | Ensemble account-code prediction and confidence                                     |
| <code>rule_matched</code>                                                                                                 | Which rule fired (null for ML rows)                                                 |
| <code>decision_path</code>                                                                                                | <code>rule</code> / <code>STANDARD</code> / <code>COMPLEX</code>                    |
| <code>xgb_pred</code> , <code>xgb_conf</code> , <code>lgb_pred</code> , <code>lgb_conf</code> , <code>models_agree</code> | Per-model trace                                                                     |
| <code>confidence_tier</code>                                                                                              | <code>RULE</code> / <code>HIGH</code> / <code>MEDIUM</code> / <code>RESOLVED</code> |
| <code>short_justification</code> , <code>full_writeup</code> , <code>audit_ready</code>                                   | Justifications and audit flag                                                       |
| <code>anomalies</code> , <code>anomaly_score</code>                                                                       | QA flags array and score                                                            |
| <code>review_priority</code> , <code>override_suggested</code> , <code>recommendation</code>                              | Review routing                                                                      |
| <code>iteration</code> , <code>processing_path</code>                                                                     | Finalization metadata                                                               |
| <code>label_source</code>                                                                                                 | <code>rule</code> / <code>ml</code> / <code>need_to_review</code>                   |
| <code>accounting_date</code>                                                                                              | Partition key                                                                       |

## 12. Configuration reference ( `atims_common.py` )

| Key                                                | Default                        | Required    | Effect                                                                                    |
|----------------------------------------------------|--------------------------------|-------------|-------------------------------------------------------------------------------------------|
| <code>CATALOG</code> , <code>STAGING_SCHEMA</code> | dev catalog                    | yes         | Where staging tables are written                                                          |
| <code>PROD_TABLE</code>                            | -                              | yes         | Final output table                                                                        |
| <code>LLM_ENDPOINT</code>                          | <code>databricks-gpt-4o</code> | yes         | Endpoint for <code>ai_query</code>                                                        |
| <code>CATEGORY_LABELS</code>                       | seed list                      | yes         | Label set for <code>ai_classify</code> - replace with the full category set               |
| <code>ACCOUNT_TO_CATEGORY</code>                   | None                           | recommended | Maps account code -> category; keeps <code>ai_query</code> cost in range (see Section 13) |
| <code>TAXABILITY_MATRIX</code>                     | -                              | yes         | Category + state -> taxability reference                                                  |
| <code>USE_TAX_COL</code>                           | <code>USE_TAX_AMOUNT</code>    | yes         | Use-tax column in the source; absent -> <code>Reverse_UseTax</code> NULL                  |
| <code>HISTORICAL_STATS</code>                      | None                           | optional    | Enables QA check 3                                                                        |
| <code>STATE_RULES</code>                           | None                           | optional    | Enables QA check 5                                                                        |
| <code>WEBHOOK_URL</code>                           | None                           | optional    | Completion notification; unset -> skipped                                                 |
| <code>THRESHOLDS["high_conf"]</code>               | 0.80                           | tunable     | HIGH vs MEDIUM tier cutoff in t32                                                         |
| <code>PATHS</code>                                 | DBFS paths                     | yes         | Source table + model artifact locations                                                   |

### 13. Processing paths & cost

LLM cost is driven by `ai_query`. The Tax Writer calls it **once per row** (unavoidable - every line gets a justification). The classifier resolver should call it only on disagreements.

| Path       | Trigger                               | ai_query calls                      |
|------------|---------------------------------------|-------------------------------------|
| FAST       | rule match                            | 1 (writer)                          |
| STANDARD   | no rule, ensemble & ai_classify agree | 1 (writer) + ai_classify            |
| COMPLEX    | no rule, disagreement                 | 2 (resolver + writer) + ai_classify |
| ESCALATION | QA anomaly or unmatched taxability    | as above + human review             |

**The cost lever.** The ensemble predicts account codes while `ai_classify` returns category names. If `ACCOUNT_TO_CATEGORY` is unset they can't match, so effectively every `no_rule` row "disagrees" and hits the resolver. Wiring the map brings resolver calls back from ~100% of `no_rule` rows to the intended ~10-15%.

### 14. Operations, monitoring & tuning

**Scheduling.** Monthly; run with parameter `invoice_month=YYYY-MM`. Order is fixed by the staging tables.

**Idempotency.** t37 writes **append** mode partitioned by `accounting_date`. Re-running a month appends duplicates - drop that month's partition first, or switch the write to merge / overwrite-by-partition.

**Re-running a stage.** Each task reads/writes its own staging table, so you can restart from any point (e.g. re-run t34->t37) without redoing t30-t33.

**MLflow.** t37 logs per run: `total`, `rule`, `ml`, `need_to_review`, `auto_pct`, plus params `invoice_month` and the model tag - your monthly coverage trend.

**Webhook.** Payload `{month, records_processed, flagged_count}`, where `flagged_count` is the rows at HIGH/CRITICAL - the handoff to the review layer.

**Tuning knobs.** `THRESHOLDS["high_conf"]` (HIGH/MEDIUM cutoff), the QA thresholds inline in `t35_qa.py` (z-score, vendor-fraction, batch-fraction), and `ACCOUNT_TO_CATEGORY` (resolver cost). The ensemble weights and TF-IDF size in `MODEL_CONFIG` only matter on a retrain.

### 15. Design decisions & known limitations

- **Existing model preserved.** The TF-IDF feature pipeline and the XGB/LGB ensemble are kept exactly. The only structural change vs. the prior notebook is the line-grain join fix in t32.
- **Account-code / category space.** The ensemble outputs account codes; `ai_classify` outputs category names. `ACCOUNT_TO_CATEGORY` reconciles them and controls resolver cost.
- **QA Check 4 dormant.** The classifier runs only on `no_rule` rows, so rule rows have no model prediction to compare against. A shadow prediction pass in t32 would activate it.
- **Use-tax NULL deferred.** `Reverse_UseTax` is NULL when `use_tax_amount` is NULL, pending confirmation of NULL-vs-data-gap semantics.
- **Train/serve population.** The model was trained on rule-matched rows but runs on `no_rule` rows; reported training accuracy reflects in-rule performance, not the served population.

### 16. Glossary

| Term                            | Meaning                                                             |
|---------------------------------|---------------------------------------------------------------------|
| <code>no_rule</code>            | A line no deterministic rule matched; goes to the ML/LLM classifier |
| <code>final_category</code>     | The resolved category used for taxability lookup and the write-up   |
| <code>label_account_code</code> | Account code assigned by a rule (the rules' label space)            |
| Ensemble                        | The 50/50 XGBoost + LightGBM probability average                    |
| <code>confidence_tier</code>    | RULE / HIGH / MEDIUM / RESOLVED - how the category was decided      |
| <code>Reverse_UseTax</code>     | Whether a use-tax reversal is required (Yes/No/null)                |
| <code>review_priority</code>    | LOW / MED / HIGH / CRITICAL - review urgency                        |
| Staging table                   | A Delta table one task writes and the next reads                    |